

LLM Foundations & Prompting Cheat Sheet

Baseline: Python 3.12, anthropic 1.8.x / openai current SDKs against claude-sonnet-5 / gpt-6-luna family, HF transformers 5.17, tiktoken 0.14.0, Ollama 0.34, Spring Boot 4.1 / Spring AI 2.0.1. Re-verified as current on 2026-09-28. Quick reference only — verify against the official docs before production use.

TOKENS & COSTS

Text is split into sub-word tokens before it reaches the model.
Roughly 3-4 chars/token in English, more per token for code and other languages.

$$cost = n_{in} \cdot p_{in} + n_{out} \cdot p_{out}$$

Count ahead of time with tiktoken, HF AutoTokenizer, or the Anthropic count_tokens endpoint.

ATTENTION FORMULA

$$Attn(Q,K,V) = softmax(QK^T / \sqrt{d_k})V$$

- Q/K/V: linear projections of the input
- $\sqrt{d_k}$ scaling keeps softmax gradients stable
- Multi-head: run several in parallel, concat, project
- MQA/GQA share K/V heads to cut KV-cache size

SAMPLING PER TASK

Task	Temp	top-p
Factual QA / extraction	0-0.2	1.0
Code generation	0.2	0.95
General chat	0.7	0.9-1.0
Creative writing	0.9-1.2	0.9-0.95
Reasoning models	ignore temp/top-p; set effort instead	

CONTEXT-WINDOW RULES

- Window = working memory; KV cache makes re-attending to it tractable per step
- "Lost in the middle": recency/primacy beat the middle of a long context
- Prompt caching reuses a static prefix — write once, read many at a discount
- Truncate or compact (summarize) once a budget is exceeded

PROMPT ANATOMY TEMPLATE

```
system: role + goals + constraints
user:
  <instructions>...</instructions>
  <context>...retrieved / background...</context>
  <input>...the actual request...</input>
  Output format: JSON matching {schema}
```

Delimiters or XML tags separate roles from data. Be specific: audience, length, format, tone, edge cases.

TECHNIQUE CATALOGUE

- Chain-of-thought:** ask for reasoning before the answer
- Self-consistency:** sample N times, take the majority
- Tree-of-thought:** branch and evaluate partial paths
- Prompt chaining:** decompose into smaller linked steps
- Self-critique:** ask the model to check its own answer
- Metaprompting:** use the model to draft/improve a prompt
- Role prompting:** assign a persona to steer style
- Prefilling:** seed the start of the assistant turn
- ReAct:** interleave reasoning with tool calls

MODEL-SELECTION FLOW

- Requirements & constraints
- ↓
- Shortlist candidates
- ↓
- Eval on your own data
- ↓
- Cost / latency check
- ↓
- Decide — or route/cascade

BENCHMARK NOTES

- MMLU-Pro / GPQA: broad / graduate-level knowledge
- SWE-bench: real GitHub issue resolution
- HumanEval+ / IFEval: code correctness / instruction-following
- MTEB: embedding-model quality
- Arena Elo: $P(i>j) = \sigma(s_i - s_j)$
- Saturation & contamination make old scores unreliable

LATENCY / COST FORMULAS

$$latency = TTFT + TPOT \cdot n_{out}$$

- TTFT: time to first token; TPOT: time per output token
- Batch APIs trade latency for a lower per-token price
- Smaller model or lower reasoning effort cuts both

STRUCTURED OUTPUT ONE-LINERS

```
Anthropic: tools=[schema], tool_choice=
{"type":"tool","name":"x"}
OpenAI: response_format=
{"type":"json_schema","json_schema":schema}
Ollama: generate(model=m, prompt=p, format=json_schema)
Spring AI:
chatClient.prompt(p).call().entity(MyRecord.class)
```

CONTEXT-ENGINEERING CHECKLIST

- Set an explicit context budget per turn
- Retrieve just-in-time, not everything up front
- Compact / summarize once nearing the budget
- Keep structured notes outside the window
- Isolate sub-tasks in sub-agents
- Order for primacy/recency, not just relevance

REFERENCE ARCHITECTURE

Context enhancement → Guardrails → Router / gateway → Caching → Agent patterns → Monitoring → Feedback loop

Every block above is covered on its own page in this sub-section — see **AI Application Architecture** for how they fit together end to end.